📖 ClauseWall — Complete Project Bible

> **Generated**: Full exhaustive codebase audit
> **Codebase**: 765 source files | 2,841-line type system | 110+ API routes | 170+ components
> **Stack**: Next.js 16 · React 19 · Supabase · Groq AI · TensorFlow.js · Tailwind CSS

Table of Contents

SECTION 1: PROJECT IDENTITY

What is ClauseWall?

ClauseWall is India's first AI-powered contract intelligence, negotiation, and legal protection platform. It combines a verified legal rules database (750+ rules across Indian statutes) with multi-model AI analysis to detect predatory, illegal, and dangerous clauses in any contract document — rental agreements, employment contracts, loan documents, NDAs, and more.

What Problem Does It Solve?

India has a massive predatory contract problem:
- Lakhs of tenants sign rental agreements with illegal security deposits (10-month deposits when the law caps at 2 months)
- Employment "bonds" with unenforceable penalty clauses trap workers
- Loan agreements with hidden escalation clauses go undetected
- Most Indians can't afford lawyer review (₹5,000–50,000 per contract)

ClauseWall democratizes legal protection by providing instant, AI-powered contract analysis with verified Indian law citations — for free.

Target Users

Segment	Use Case
Tenants	Rental agreement review before signing
Job seekers	Employment offer/bond analysis
Small businesses	Vendor/service contract verification
Freelancers	Client contract clause checking
Loan applicants	Loan agreement predatory clause detection

| **Consumer advocates** | Tracking corporate ToS changes |

What Makes It Unique vs Competitors

Feature	ChatGPT/Generic AI	LegalTech SaaS	**ClauseWall**
Indian law specificity	❌ Generic	⚠️ Some	✅ 750+ verified rules
Exact statute citations	❌ Hallucinated	⚠️ Manual	✅ DB-verified + AI
Free to use	❌ Paid	❌ Paid	✅ 100% free
Negotiation scripts	❌	❌	✅ Counter-response playbooks
Browser extension	❌	❌	✅ Auto-scan any ToS page
Community intelligence	❌	❌	✅ Crowdsourced pattern DB
Multilingual support	⚠️	❌	✅ 10+ Indian languages

Core Value Proposition

"Know your rights BEFORE you sign." — ClauseWall turns every Indian into a legally-aware consumer by making contract analysis instant, free, and actionable.

Emotional Hook

"That clause on page 7 that says you lose ₹1.5 lakhs if you leave early? It's illegal. The Indian Contract Act, Section 74 says so. Here's exactly what to say to your landlord."

One-Line Pitch

> AI-powered contract analyzer that finds predatory clauses, cites exact Indian laws they violate, and generates negotiation scripts — for free.

Elevator Pitch (30 seconds)

> Every year, millions of Indians sign contracts with illegal clauses — and they don't know it. ClauseWall is India's first AI contract analyzer. Upload your rental agreement, employment offer, or loan document. In 5 seconds, you get a risk score. In 60 seconds, you get clause-by-clause analysis with exact Indian law citations, negotiation scripts, and a legal notice generator. It's 100% free, no signup required, and works in 10+ Indian languages. We've built a verified database of 750+ legal rules, a browser extension, a Telegram bot, and community intelligence that gets smarter with every contract analyzed.

SECTION 2: COMPLETE TECH STACK

Core Framework

Technology	Version	Role	Why Chosen
Next.js	16 (canary)	Full-stack framework	App Router, API routes, SSR/SSG, edge runtime
React	19	UI rendering	Latest with concurrent features
TypeScript	5.x	Type safety	Strict mode enabled, 2841-line type system
Tailwind CSS	4.x	Styling	Rapid prototyping, responsive design

AI & ML

Technology	Role	Integration Point
Groq SDK (`groq-sdk`)	Primary AI inference	`lib/ai/groq-client.ts` — 3-key rotation
Google Generative AI (`@google/generative-ai`)	Secondary AI (Gemini)

`lib/bot/gemini-client.ts` — 3-key rotation |
TensorFlow.js (`@tensorflow/tfjs`)	On-device ML classification	`lib/ml/` — browser-side clause scoring
Tesseract.js	OCR for scanned contracts	`lib/ocr/index.ts`
Whisper (via Groq)	Audio transcription	`lib/negotiate/whisper-client.ts`

Database & Auth

Technology	Role	Files
Supabase (PostgreSQL)	Primary database, auth, RLS, storage	`lib/supabase/{client,server,admin,middleware}.ts`
Supabase Auth	User authentication	`middleware.ts` (currently auth protection commented out)
Supabase Storage	Evidence file storage	`lib/evidence/storage.ts`

PDF & Document Processing

Technology	Role
pdfjs-dist	Client-side PDF parsing
unpdf	Server-side PDF text extraction
jsPDF + **jspdf-autotable**	PDF report generation

UI Libraries

Library	Role
Radix UI	Accessible component primitives (dialog, dropdown, tabs, etc.)
Framer Motion	Animations & transitions
Lucide React	Icon system
Sonner	Toast notifications
react-dropzone	File upload dropzone
Recharts	Charts and data visualization
react-simple-maps	Geographic heat maps

Utilities & Infrastructure

Library	Role
crypto (Node.js)	SHA-256 hashing for evidence chains
QRCode	QR badge generation
html-to-image	Score card & video card export
Resend	Email notifications
ical-generator	Calendar export (.ics) for deadlines
web-push	Push notifications
class-variance-authority	Component variant styling
clsx + **tailwind-merge**	Utility CSS merging

Environment Variables (16 total)

Variable	Purpose
`NEXT_PUBLIC_SUPABASE_URL`	Supabase project URL
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	Supabase anonymous key
`SUPABASE_SERVICE_ROLE_KEY`	Supabase admin operations
`GROQ_API_KEY_1`, `_2`, `_3`	Groq AI (3-key rotation for rate limits)
`GOOGLE_AI_API_KEY_1`, `_2`, `_3`	Gemini AI (3-key rotation)
`TELEGRAM_BOT_TOKEN`	Telegram bot integration
`HUGGINGFACE_API_KEY`	HuggingFace embeddings
`RESEND_API_KEY`	Email service
`VAPID` keys (public + private)	Web push notifications

```
| `NEXT_PUBLIC_APP_URL` | Application URL |

---

## SECTION 3: ARCHITECTURE OVERVIEW

### High-Level Architecture

```mermaid
graph TB
    subgraph Client["Browser Client"]
        LP[Landing Page]
        UP[Upload Page]
        RP[Results Page]
        EXT[Chrome Extension]
        ML[TensorFlow.js On-Device ML]
    end

    subgraph API["Next.js API Routes (110+)"]
        AN["/api/analyze"]
        QS["/api/quick-scan"]
        TA["/api/bot/trigger-analysis"]
        AI_ROUTES["AI Feature Routes"]
    end

    subgraph Core["Core Analysis Engine"]
        AZ["analyzer.ts (Orchestrator)"]
        HA["hybrid-analyzer.ts"]
        RE["rule-engine.ts"]
        CE["clause-extractor.ts"]
        SC["scorer.ts"]
        LM["legal-matcher.ts"]
        EE["entity-extractor.ts"]
    end

    subgraph AI["AI Layer"]
        GC["groq-client.ts (3-key rotation)"]
        GM["gemini-client.ts (3-key rotation)"]
        SP["system-prompt.ts (737 lines)"]
    end

    subgraph DB["Supabase PostgreSQL"]
        DOC["documents"]
        CL["clauses"]
        SR["structured_rules"]
        LR["legal_rules"]
        FE["flagged_entities"]
        CC["community_clauses"]
        EC["evidence_cases"]
        MC["monitored_companies"]
    end

    UP --> QS
    UP --> AN
    AN --> TA
    TA --> AZ
    AZ --> CE
    AZ --> HA
    HA --> RE
    HA --> GC
    RE --> SR
    AZ --> SC
    AZ --> LM
    LM --> LR
```
```

```
    AZ --> DOC
    AZ --> CL
    EXT --> QS
    ML --> UP
```

Request Flow

```
User Upload → /api/analyze (create document record)
           → /api/bot/trigger-analysis (async)
           → analyzeDocument() orchestrator
              ├─ Language Detection
              ├─ Clause Extraction (AI)
              ├─ For each clause:
              │   ├─ Extract structured values (AI)
              │   ├─ Match against structured_rules DB
              │   ├─ If DB match → verified result
              │   ├─ If no DB match → AI analysis (fallback)
              │   ├─ Neurosymbolic proof tree
              │   └─ Community intelligence check
              ├─ Knowledge Graph enrichment
              ├─ Power Balance analysis
              ├─ Risk scoring (weighted algorithm)
              ├─ Summary generation
              ├─ Blockchain proofing (SHA-256 + TSA)
              ├─ State Machine analysis
              ├─ Temporal deadline extraction
              └─ Poison Pill detection
           → Update document status → "completed"
           → Client polls → redirect to /results/[id]
```

Directory Structure

```
clausewall/
├── app/                          # Next.js App Router
│   ├── api/                      # 110+ API routes
│   │   ├── analyze/              # Main analysis endpoint
│   │   ├── quick-scan/           # 5-second quick scan
│   │   ├── bot/                  # Telegram bot webhooks
│   │   ├── collective/           # Collective bargaining
│   │   ├── complaint/            # Regulatory complaints
│   │   ├── deliberation/         # Multi-agent AI debate
│   │   ├── evidence/             # Evidence chain builder
│   │   ├── graph/                # Knowledge graph
│   │   ├── lawchange/            # Legal change tracker
│   │   ├── market/               # Market intelligence
│   │   ├── negotiate/            # Live negotiation
│   │   ├── poisonpill/           # Poison pill detection
│   │   ├── reasoning/            # Neurosymbolic proofs
│   │   ├── ruin-calculator/      # Monte Carlo simulation
│   │   ├── shadow/               # Shadow agreement detector
│   │   ├── statemachine/         # Contract state machine
│   │   ├── timebomb/             # Deadline tracker
│   │   ├── vault/                # Multi-contract vault
│   │   └── watchdog/             # ToS change monitor
│   ├── analyze/[id]/             # Analysis progress page
│   ├── results/[id]/             # Results page (1335 lines)
│   ├── upload/                   # Upload page (845 lines)
│   ├── dashboard/                # User dashboard
│   ├── builder/                  # Contract builder
│   ├── watchdog/                 # ToS Watchdog UI
```

```
│   ├── evidence/                # Evidence workspace
│   └── ... (57 total page routes)
├── components/                  # 170+ React components
│   ├── ui/                      # Radix UI primitives (17)
│   ├── results/                 # Analysis result views (30+)
│   ├── upload/                  # Upload flow components
│   ├── watchdog/                # ToS watchdog components
│   ├── negotiate/               # Negotiation components
│   └── ... (15+ feature folders)
├── lib/                         # Core business logic
│   ├── ai/                      # AI clients & prompts
│   ├── core/                    # Analysis engine
│   ├── bot/                     # Telegram bot logic
│   ├── ml/                      # TensorFlow.js models
│   └── ... (25+ feature libraries)
├── types/                       # TypeScript definitions (7 files)
├── extension/                   # Chrome extension
├── scripts/                     # Migration & utility scripts
└── public/                      # Static assets
```

SECTION 4: DATABASE SCHEMA

Core Tables

`documents` — Analyzed contracts
Column	Type	Description
`id`	UUID PK	Document identifier
`user_id`	UUID FK (nullable)	Owner (null = anonymous)
`original_filename`	text	Upload filename
`document_type`	enum	rental/employment/tos/loan/freelance/sale/partnership/nda/other
`jurisdiction`	text	Indian state code
`raw_text`	text	Full extracted text
`overall_risk_score`	int	0-100 weighted score
`total_clauses`	int	Count of extracted clauses
`safe_count`, `warning_count`, `dangerous_count`, `illegal_count`	int	Risk breakdown
`entity_name`	text	Identified counterparty
`summary`	text	AI-generated summary
`analysis_status`	enum	pending/analyzing/completed/failed
`power_balance`	jsonb	Power asymmetry data
`proof_hash`, `proof_cid`, `proof_timestamp`	text	Blockchain proof data
`state_machine_data`	jsonb	Contract state machine
`deliberation_data`	jsonb	Multi-agent debate results
`temporal_data`	jsonb	Deadline/temporal data
`poison_pill_data`	jsonb	Poison pill analysis
`shadow_analysis_data`	jsonb	Shadow agreement data
`source_language`, `detected_language`	text	Bhasha engine data

`clauses` — Individual extracted clauses
Column	Type	Description
`id`	UUID PK	Clause identifier
`document_id`	UUID FK	Parent document
`clause_number`	int	Sequence number
`original_text`	text	Clause text
`clause_type`	text	security_deposit/lock_in/notice_period/etc.
`risk_level`	enum	safe/warning/dangerous/illegal
`risk_score`	int	0-100
`explanation`	text	Human-readable explanation

`legal_issue`	text	Legal violation description
`legal_citation`	text	Applicable law
`statute_code`	text	Formal statute reference
`fair_alternative`	text	Fair clause suggestion
`red_flags`	text[]	Array of red flag descriptions
`extracted_value`	numeric	Extracted numeric value
`extracted_unit`	text	Unit (months/days/rupees)

`structured_rules` — Verified legal rules database (THE MOAT)

Column	Type	Description
`id`	UUID PK	Rule identifier
`clause_type`	text	Matches clause_type
`jurisdiction`	text	State code or "central"
`document_type`	text	rental/employment/etc.
`rule_type`	enum	max_value/min_value/prohibited/required/must_be_mutual
`limit_value`	numeric	Legal limit (e.g., 2 for max 2 months deposit)
`limit_unit`	text	Unit for limit
`statute_name`	text	Full statute name
`statute_code`	text	Formal citation
`severity`	enum	Risk level for violations
`violation_template`	text	Template with {value} placeholders
`fair_alternative`	text	Model clause text
`negotiation_script`	text	What to say to counterparty
`keywords`	text[]	Matching keywords

`legal_rules` — Legacy legal reference database

Column	Type	Description
`id`	UUID PK	Rule identifier
`jurisdiction`	text	State
`document_type`	text	Contract type
`clause_type`	text	Clause category
`rule_title`	text	Human-readable title
`statute_code`	text	Formal citation
`keywords`	text[]	Search keywords

Community & Social Tables

- **`flagged_entities`** — Crowdsourced entity reputation (landlords, companies)
- **`community_clauses`** — Anonymized predatory clause patterns
- **`reports`** — User-submitted violation reports

Evidence Chain Tables (4 tables)

- **`evidence_cases`** — Dispute case containers
- **`evidence_items`** — Individual evidence pieces with SHA-256 chain hashing
- **`evidence_certificates`** — Section 65B certificates for court admissibility
- **`evidence_bundles`** — Generated evidence bundle PDFs

Watchdog Tables (6 tables)

- **`monitored_companies`** — Companies tracked for ToS changes
- **`tos_snapshots`** — Historical ToS text snapshots
- **`tos_changes`** — Detected changes with AI classification
- **`user_watchlist`** — User subscription to company alerts
- **`watchdog_alerts`** — Generated alerts
- **`optout_campaigns`** + **`campaign_signatories`** — Collective action

RLS Policies

All tables have Row Level Security enabled. Key policies:
- Evidence tables: `auth.uid() = user_id` (users access own data only)
- Watchdog public tables: `FOR SELECT USING (true)` (public read)

- Watchdog user tables: `auth.uid() = user_id` (own data only)

> [!WARNING]
> **Auth protection is currently commented out in `middleware.ts`** — all API routes are effectively public. The `user_id` field is set to `null` for anonymous analysis.

SECTION 5: CORE ANALYSIS PIPELINE

Orchestrator: `lib/core/analyzer.ts` (647 lines)

The `analyzeDocument()` function is the central entry point. It manages a 12-step pipeline:

Step	Function	Description	Failure Handling
1	Language Detection	Detect source language via Bhasha	Non-blocking
2	Clause Extraction	AI splits document into clauses	**Critical** — fails entire analysis
3	Hybrid Analysis	DB-first + AI fallback per clause	Per-clause error isolation
4	Community Intelligence	Check clause against community DB	Non-blocking
5	Knowledge Graph	Build clause relationship graph	Non-blocking
6	Power Balance	Assess party power asymmetry	Non-blocking
7	Risk Scoring	Weighted scoring algorithm	Always runs
8	Summary Generation	AI-generated plain English summary	Non-blocking
9	Blockchain Proofing	SHA-256 hash + timestamp	Non-blocking
10	State Machine	Model contract lifecycle states	Non-blocking
11	Temporal Analysis	Extract deadlines and dates	Non-blocking
12	Poison Pill Detection	Find hidden trap clauses	Non-blocking

Scoring Algorithm: `lib/core/scorer.ts` (127 lines)

```
Weighted Score = Σ(clause_score × severity_weight) / Σ(severity_weight)

Severity weights:
  illegal    = 3.0×
  dangerous  = 2.0×
  warning    = 1.5×
  safe       = 1.0×
```

Hybrid Analyzer: `lib/core/hybrid-analyzer.ts` (258 lines)

The **key innovation** — deterministic DB rules take priority over AI:

```
1. Extract structured values from clause (AI lightweight extraction)
2. Query structured_rules DB for matching rules
3. IF DB match found:
   → Use verified rule data (statute, severity, alternative)
   → Confidence = "verified"
4. ELSE:
   → Fall back to full AI analysis
   → Cross-reference with legal_rules DB
   → Confidence = "ai_suggested"
5. Apply neurosymbolic proof tree
6. Return HybridAnalysisResult
```

Rule Engine: `lib/core/rule-engine.ts` (302 lines)

Pure logic engine supporting rule types:
- `max_value` — e.g., security deposit ≤ 2 months rent
- `min_value` — e.g., notice period ≥ 30 days
- `prohibited` — e.g., non-compete for non-senior employees
- `must_be_mutual` — e.g., termination rights
- Unit conversion support (days ↔ months)

SECTION 6: AI & ML LAYER

Groq Client: `lib/ai/groq-client.ts` (435 lines)

3-key automatic rotation for rate limit management:
- Cycles through `GROQ_API_KEY_1`, `_2`, `_3`
- Auto-retries on 429 (rate limit) with next key
- Exponential backoff on failures
- Default model: `llama-3.3-70b-versatile`
- Also supports Whisper audio transcription

Gemini Client: `lib/bot/gemini-client.ts`

Secondary AI with similar 3-key rotation for Google's Gemini models.

System Prompts: `lib/ai/system-prompt.ts` (737 lines)

Defines structured JSON output formats for:
- Clause extraction (types, parties, dates)
- Clause analysis (risk level, legal issue, alternative)
- Demand letter generation
- Clause autopsy (word-by-word violation mapping)
- Roast mode (humorous commentary)
- Escape planning
- Contract simulation

On-Device ML: `lib/ml/` (6 files)

TensorFlow.js-based client-side classification:
- `model-loader.ts` — Downloads & caches model
- `classifier.ts` — Runs inference in browser
- `preprocessor.ts` — Text → tensor conversion
- `clause-splitter.ts` — Regex-based clause splitting
- Provides instant (<100ms) preliminary results before server analysis

---

SECTION 7: API ROUTES MAP

110+ API routes organized by feature:

Feature Area	Routes	Key Endpoints
Core Analysis	4	`/api/analyze`, `/api/quick-scan`, `/api/reanalyze`, `/api/verify-clauses`
Clause Actions	4	`/api/roast`, `/api/rewrite`, `/api/explain`, `/api/simulate`
Negotiation	7	`/api/negotiate/generate`, `/api/negotiate/live/*` (transcribe, counter-argument, bluff-check, camera-ocr, lookup)
Legal Actions	2	`/api/generate-letter`, `/api/escape`
Community	2	`/api/community/check`, `/api/flag-entity`
Deliberation	3	`/api/deliberation/run`, `/api/deliberation/single`, `/api/deliberation/[documentId]`

Reasoning	4	`/api/reasoning/prove`, `/api/reasoning/challenge`, `/api/reasoning/explain-step`, `/api/reasoning/proof/[clauseId]`
State Machine	4	`/api/statemachine/extract`, `/api/statemachine/simulate`, `/api/statemachine/path`, `/api/statemachine/[documentId]`
Timebomb	8	Calendar, notifications, reminders, defuse, activate, action-letter
Poison Pill	2	`/api/poisonpill/[documentId]`, `/api/poisonpill/reanalyze`
Shadow Agreement	4	`/api/shadow/analyze`, `/api/shadow/parse-preview`, `/api/shadow/[documentId]`, `/api/shadow/report/[documentId]`
Evidence Chain	11	Cases CRUD, capture (audio/email/receipt/url/whatsapp), bundle, certificate, chain verify
Complaint Filing	7	Generate, authorities, determine, fee, filing-guide, hearing-prep, list
Watchdog	11	Companies, changes, campaigns, alerts, leaderboard, scrape, watchlist
Law Change	7	Recent, impacts, retroactive, notifications, pending, summary
Market Intelligence	8	Benchmarks, geographic, trends, compare, ammunition, narrative, stats
Ruin Calculator	2	`/api/ruin-calculator`, `/api/ruin-calculator/stress-test`
Contract Vault	5	Analyze, documents, latest, whatif, [analysisId]
Collective	3	Legal-aid, notifications, user
Voice Aid	4	Process, session, tts, languages
Graph	4	Clause, cases, explore, traverse
Verification	1	`/api/verify/generate`
Extension	1	`/api/extension/analyze`
Cron	2	`/api/cron/daily`, `/api/cron/deadlines`
Testing	1	`/api/test-gemini`

SECTION 8: FEATURE MODULES (Deep Dive)

8.1 Quick Scan Engine
- **Files**: `lib/bot/quick-analyzer.ts`, `components/upload/quick-scan-result.tsx`
- **Purpose**: 5-second AI scan for immediate red flag detection
- **Flow**: Text → Groq AI → Risk score + top 5 issues → Display

8.2 ML Instant Scan (On-Device)
- **Files**: `lib/ml/` (6 files), `components/upload/ml-instant-result.tsx`
- **Purpose**: Sub-100ms classification using TensorFlow.js in-browser
- **Privacy**: Maximum privacy mode — zero data leaves device

8.3 Hybrid Analysis Engine
- **Files**: `lib/core/hybrid-analyzer.ts`, `lib/core/rule-engine.ts`
- **Purpose**: DB-verified rules + AI fallback for comprehensive analysis
- **Key Innovation**: Deterministic legal accuracy > AI suggestions

8.4 Neurosymbolic Reasoning
- **Files**: `lib/reasoning/` (6 files: engine, fact-bridge, proof-formatter, rule-compiler, types)
- **Purpose**: Formal proof trees for each clause violation
- **UI**: `components/results/proof-tree.tsx`, `proof-walkthrough.tsx`

8.5 Multi-Agent Deliberation
- **Files**: `lib/deliberation/` (4 files: engine, prompts, types, index)
- **Purpose**: 3 AI agents debate each clause: Predator Finder, Devil's Advocate, Judge
- **UI**: `components/deliberation/document-deliberation.tsx`

8.6 Negotiation Playbook
- **Files**: `lib/negotiate/` (8 files: audio-processor, bluff-checker, camera-processor, pressure-tactics, quick-lookup, session-manager, tts-engine, whisper-client)
- **Purpose**: Real-time negotiation assistance with live audio/camera
- **UI**: 7 components including floating lookup bar, camera scanner, tactic alerts

8.7 Contract Simulator
- **Files**: `lib/simulation/` (8 files: clause-cost-engine, fair-comparison, formatters, monte-carlo, probability, risk-adjusted, stress-test, types)
- **Purpose**: Financial impact modeling — "What will this contract cost you?"

8.8 Ruin Calculator
- **Files**: `lib/simulation/monte-carlo-engine.ts`, `stress-test-engine.ts`
- **Purpose**: Monte Carlo simulation of cascading financial catastrophes
- **UI**: 8 components (monte-carlo-chart, stress-test-builder, insurance-gap-meter, etc.)

8.9 Shadow Agreement Detector
- **Files**: `lib/shadow/` (9 files: shadow-engine, promise-extractor, mismatch-detector, legal-analyzer, report-generator, parsers/*)
- **Purpose**: Detect contradictions between informal promises and formal contracts
- **Input**: WhatsApp chats, emails, audio recordings vs. written contract

8.10 Contract State Machine
- **Files**: `lib/statemachine/` (5 files: analyzer, extractor, templates, types, index)
- **Purpose**: Model contract as finite state machine — find trap states
- **UI**: `components/statemachine/state-graph.tsx` (visual FSM), timeline-slider

8.11 Poison Pill Detector
- **Files**: `lib/poisonpill/` (5 files: trap-detector, trap-patterns, pattern-matcher, graph-builder, index)
- **Purpose**: Find interconnected clause traps that work together
- **UI**: `components/poisonpill/interconnection-map.tsx`, trap-mechanism-flow

8.12 Timebomb Tracker
- **Files**: `lib/timebomb/` (7 files: temporal-extractor, date-calculator, ics-generator, push-registration, reminder-service, action-generator, index)
- **Purpose**: Extract all deadlines, generate calendar events, push reminders

8.13 Evidence Chain Builder
- **Files**: `lib/evidence/` (10 files: capture, chain, storage, certificate-template, section-65b-generator, evidence-bundle-generator, parsers/*)
- **Purpose**: Build court-admissible evidence bundles with SHA-256 chain hashing
- **Legal**: Section 65B Indian Evidence Act certificate auto-generation

8.14 Regulatory Complaint Filing
- **Files**: `lib/complaint/` (8 files: authority-router, complaint-generator, case-tracker, hearing-prep, jurisdiction-resolver, fee-calculator, filing-guide, prompts)
- **Purpose**: Auto-generate and file complaints with Indian consumer forums

8.15 Contract Watchdog (ToS Monitor)
- **Files**: `lib/watchdog/` (13 files: scraper, snapshot-manager, semantic-diff, change-classifier, legality-checker, alert-dispatcher, campaign-manager, company-registry, cron-handler, score-calculator, text-cleaner, prompts, types)
- **Purpose**: Monitor corporate Terms of Service for changes, classify impact
- **UI**: 15 components (company grid, change feed, campaign cards, leaderboard)

8.16 Law Change Tracker
- **Files**: `lib/lawchange/` (8 files: scrapers/*, classifier, impact-analyzer, retroactive-analyzer, notification-sender, db, index)
- **Purpose**: Scrape Indian legal sources for new laws, analyze retroactive impact on existing contracts

8.17 Market Intelligence
- **Files**: `lib/market/` (10 files: aggregator, ammunition, benchmarks, comparator, constants, geographic, narrative, percentile, scheduler, trends)
- **Purpose**: Benchmark contract terms against market data — "90% of Mumbai rentals have lower deposits"

8.18 Contract Vault
- **Files**: `lib/vault/` (7 files: cascade-analyzer, conflict-detector, exposure-calculator, gap-analyzer, obligation-unifier, vault-scorer, whatif-simulator)
- **Purpose**: Multi-contract portfolio analysis — find conflicts, gaps, cascading risks

8.19 Bhasha Engine (Multilingual)
- **Files**: `lib/bhasha/`, `lib/voice-aid/`, `components/bhasha/`, `components/voice-aid/`
- **Purpose**: 10+ Indian language support — Hindi, Marathi, Tamil, Telugu, Bengali, Gujarati, Kannada, Malayalam, Punjabi, Odia
- **Features**: Auto-detection, bilingual toggle, audio player, terminology tooltips

8.20 Privacy Engine
- **Files**: `lib/privacy/` (4 files: pii-detector, redactor, types, index)
- **Purpose**: 3-tier privacy (standard/balanced/maximum) with PII redaction
- **UI**: Privacy toggle, pre-send review modal, privacy dashboard

8.21 Knowledge Graph
- **Files**: `lib/graph/` (4 files: graph-engine, graph-query, types, index)
- **Purpose**: Clause relationship visualization and traversal

8.22 Contract Builder
- **Files**: Template configs in types + API routes
- **Purpose**: Generate fair contracts from templates with Indian law compliance

8.23 Additional Features
- **Clause Autopsy** — Word-by-word violation mapping
- **Roast Mode** — Humorous take on predatory clauses
- **Escape Plan** — Step-by-step exit strategy with recovery estimation
- **QR Verification Badge** — "Scan Before You Sign"
- **Contract DNA** — Visual fingerprint (6 styles: constellation, helix, heartbeat, skyline, waveform, fingerprint)
- **Mood Ring Background** — UI background changes based on active clause risk
- **Score Card & Video Card** — Shareable analysis cards
- **Voice Commands** — Voice-controlled interface
- **X-Ray Mode** — See through legalese overlay
- **Wall of Shame** — Public entity reputation

SECTION 9: UI COMPONENTS

Component Count by Feature Area

Area	Count	Key Components
`ui/`	17	button, card, dialog, select, tabs, badge, skeleton, etc.
`results/`	30+	clause-card, danger-gauge, proof-tree, mood-ring-background, xray-mode, etc.

`upload/`	8	dropzone, quick-scan-result, ml-instant-result, privacy-toggle, etc.
`watchdog/`	15	company-grid, change-feed, campaign-card, leaderboard-table, etc.
`negotiate/`	7	negotiation-companion, camera-scanner, floating-lookup, tactic-alert, etc.
`timebomb/`	8	countdown-widget, deadline-card, calendar-export, signing-date-modal, etc.
`poisonpill/`	7	interconnection-map, trap-mechanism-flow, trap-card, negotiation-roadmap, etc.
`shadow/`	8	evidence-upload, mismatch-card, promise-timeline, trust-score-gauge, etc.
`ruin-calculator/`	8	monte-carlo-chart, stress-test-builder, insurance-gap-meter, etc.
`vault/`	9	contract-selector, exposure-dashboard, obligations-list, whatif-panel, etc.
`evidence/`	6	evidence-workspace, capture-modal, chain-timeline, certificate-preview, etc.
`voice-aid/`	7	voice-interface, microphone-button, audio-waveform, language-selector, etc.
`statemachine/`	6	state-graph, timeline-slider, trap-state-card, report-card, etc.
Other areas	30+	complaint, collective, authority, bhasha, market, deliberation, etc.

Total: ~170+ React components

SECTION 10: BROWSER EXTENSION

Architecture

- **Manifest V3** Chrome extension
- **Content Script** (`content.js`, 533 lines): Injects into all pages
- **Background Worker** (`background.js`): Manages API communication
- **Popup** (`popup/popup.html`): Extension popup UI

Detection Pipeline

```
Page Load → URL Pattern Check → Title Pattern Check → Content Marker Check
  → If ToS detected: Extract page text → Send to ClauseWall API
  → Receive analysis → Highlight dangerous clauses inline
  → Show floating badge with risk score
  → Hover tooltips on highlighted clauses
```

Key Features
- Auto-detects Terms of Service pages (12 URL patterns, 9 title patterns, 13 content markers)
- Inline clause highlighting with 3 severity colors
- Floating risk badge with pulse animation for dangerous pages
- Click-to-expand tooltip with risk explanation and legal citations
- Works on Chrome, Brave, Edge

Host Permissions
- `https://clause-wall.vercel.app/*`
- `http://localhost:3000/*`

SECTION 11: TELEGRAM BOT

Files
- `lib/bot/telegram-client.ts` — Bot API wrapper
- `lib/bot/quick-analyzer.ts` — Quick analysis for bot context
- `lib/bot/gemini-client.ts` — Gemini AI client (3-key rotation)
- `lib/bot/format-response.ts` — Telegram message formatting
- `lib/bot/compare-analyzer.ts` — Contract comparison
- `app/api/bot/` — Webhook routes

Capabilities
- Photo/document upload → OCR → Analysis
- Text paste → Quick scan
- Voice message → Whisper → Analysis
- Multi-language support via Bhasha engine
- Inline result formatting with emojis

SECTION 12: SECURITY AUDIT

🔴 Critical Issues

Issue	Severity	Location	Impact
Auth middleware disabled	🔴 CRITICAL	`middleware.ts:24-42`	All routes public, no user isolation
API keys in .env.local	🔴 CRITICAL	`.env.local`	16 API keys including Supabase service role key
Service role key exposed	🔴 CRITICAL	`.env.local:4`	Full database admin access
Telegram bot token exposed	▫ HIGH	`.env.local:15`	Bot can be impersonated

▫ Medium Issues

Issue	Severity	Description
No rate limiting	▫ HIGH	API routes have no request rate limiting
No input sanitization	▫ MEDIUM	Raw text stored without XSS sanitization
Anonymous analysis	▫ MEDIUM	`user_id: null` for all analyses — no accountability
CORS not explicitly configured	▫ MEDIUM	Relies on Next.js defaults

✅ Good Practices

- RLS enabled on all tables
- Supabase separate client/server/admin instances
- Entity extraction has hallucination rejection
- Privacy engine with PII redaction
- Evidence chain uses SHA-256 cryptographic hashing

> [!CAUTION]
> **The `.env.local` file contains live production keys (Supabase, Groq, Gemini, Telegram, Resend, HuggingFace, VAPID).** These should NEVER be committed to version control. Verify `.gitignore` includes `.env.local`.

SECTION 13: PERFORMANCE ANALYSIS

Bottlenecks

Operation	Current Latency	Target	Issue

Quick Scan	3-5 sec	2 sec	Single AI call, acceptable
Full Analysis	30-90 sec	30 sec	12-step sequential pipeline
Clause Extraction	5-10 sec	3 sec	Single large AI call
Per-Clause Analysis	2-4 sec × N clauses	1 sec	Currently sequential
ML Instant Scan	<100ms	<50ms	TF.js model size
Extension Analysis	5-10 sec	3 sec	Network-bound

Optimization Opportunities

1. **Parallelize clause analysis** — Currently sequential; batch or parallelize per-clause hybrid analysis
2. **Cache structured_rules lookups** — Rules don't change often; add in-memory cache
3. **Stream results** — Use SSE/WebSocket instead of polling for analysis progress
4. **Prune non-blocking steps** — Skip graph/power-balance/poison-pill for simple contracts
5. **Model quantization** — Reduce TF.js model size for faster loading

Resource Usage

- **API Key Rotation**: 3 Groq keys + 3 Gemini keys mitigate rate limits
- **Max execution time**: 60 seconds (`maxDuration = 60` in analyze route)
- **Text truncation**: 15,000 chars max for clause extraction
- **File size limit**: 10MB max upload

SECTION 14: DEPLOYMENT

Platform: Vercel

- **Framework**: Next.js with App Router
- **Runtime**: Node.js (not Edge — required for PDF parsing)
- **Cron**: Daily job at 2:30 AM UTC (`/api/cron/daily`)
- **Config**: `vercel.json` defines cron schedule only
- **URL**: `https://clause-wall.vercel.app`

Build Configuration

```json
// next.config.ts
{
  turbopack: {},
  webpack: {
    // Fix pdfjs-dist canvas issue
    alias: { canvas: false },
    // Allow .mjs worker files
    rules: [{ test: /pdf\.worker\.mjs$/, type: "asset/resource" }]
  }
}
```

TypeScript Configuration

- Strict mode enabled
- Module resolution: bundler
- JSX: react-jsx
- Path alias: `@/*` → `./*`
- Includes: `*.ts`, `*.tsx`, `*.mts`, `types/**/*.d.ts`

SECTION 15: DATA FLOW DIAGRAMS

Upload → Analysis → Results Flow

```mermaid
sequenceDiagram
    participant U as User
    participant UP as Upload Page
    participant ML as TF.js (Browser)
    participant QS as /api/quick-scan
    participant AN as /api/analyze
    participant TA as /api/bot/trigger-analysis
    participant AZ as analyzeDocument()
    participant DB as Supabase

    U->>UP: Upload file/paste text
    UP->>ML: Instant ML scan (on-device)
    ML-->>UP: MLScanResult (<100ms)
    UP->>QS: Quick scan request
    QS-->>UP: QuickAnalysisResult (3-5s)
    UP->>AN: Create document record
    AN->>DB: INSERT into documents
    AN-->>UP: { documentId, status: "analyzing" }
    UP->>TA: Trigger full analysis (async)
    TA->>AZ: analyzeDocument(text, docType, jurisdiction)
    AZ->>DB: UPDATE documents SET status='analyzing'
    loop For each clause
        AZ->>AZ: Hybrid analysis (DB → AI fallback)
        AZ->>DB: INSERT into clauses
    end
    AZ->>DB: UPDATE documents SET status='completed'
    U->>UP: Redirect to /results/[id]
    Note over UP: Polls DB every 3 seconds until completed
```

Evidence Chain Flow

```
Upload Evidence → Parse (OCR/transcribe/extract)
  → Compute SHA-256 content hash
  → Chain with previous item's hash
  → Store in evidence_items
  → Generate Section 65B certificate
  → Bundle into court-ready PDF
```

SECTION 16: KNOWN ISSUES & BUGS

#	Issue	Severity	Location	Status
1	Auth middleware commented out	🔴 Critical	`middleware.ts`	Known - intentional for hackathon
2	MLInstantResult component crash potential	▫ Medium	`components/upload/ml-instant-result.tsx`	Identified in past conversation
3	ContractDNAPreview data propagation	▫ Medium	`components/results/contract-dna-preview.tsx`	Identified in past conversation
4	Shadow Agreement route was 404	▫ Fixed	`app/shadow/page.tsx`	Fixed in conversation `72310c1b`
5	Some API routes have mangled filenames	▫ Medium	e.g., `route.tse.ts`, `route.tse\\route.ts`	File system artifacts
6	Non-fatal errors in enrichment steps silently caught	▫ Low	`lib/core/analyzer.ts`	By design but needs monitoring

| 7 | Community embeddings may timeout | ▫ Low | `lib/community/embedder.ts` | HuggingFace dependency |
| 8 | PDF worker path may fail in production | ▫ Medium | `next.config.ts` | Webpack alias workaround |

> [!NOTE]
> Item #5 — Several API route files appear to have corrupted filenames (e.g., `route.tse.ts` instead of `route.ts`). These may be file system artifacts from Windows and should be verified/cleaned.

SECTION 17: PRODUCTION READINESS SCORECARD

Category	Score	Details
Core Functionality	✅ 9/10	Hybrid analysis pipeline is robust and well-architected
Type Safety	✅ 9/10	2,841-line type system, strict mode, comprehensive interfaces
Error Handling	⚠️ 7/10	Good per-step isolation but some silent failures
Authentication	❌ 3/10	Middleware exists but auth protection disabled
Authorization	⚠️ 5/10	RLS policies exist but `user_id` is null
Rate Limiting	❌ 2/10	No API rate limiting, only AI key rotation
Input Validation	⚠️ 6/10	Basic validation exists, entity anti-hallucination is excellent
Security	❌ 4/10	API keys exposed, no CSRF, no rate limiting
Performance	⚠️ 6/10	Sequential pipeline; needs parallelization
Testing	❌ 1/10	No unit tests, integration tests, or E2E tests
Monitoring	❌ 2/10	Console.log only, no structured logging or APM
Documentation	⚠️ 5/10	Good inline comments but no external docs
Deployment	✅ 8/10	Vercel with cron, clean config
Feature Completeness	✅ 10/10	Extraordinarily feature-rich (30+ modules)
UI/UX	✅ 9/10	Premium dark theme, animations, accessibility

Overall: 6.1/10 — Feature-complete and architecturally sound, but needs security hardening, testing, and monitoring for production deployment.

SECTION 18: ACTIONABLE RECOMMENDATIONS

🔴 Priority 1: Security (Do First)

1. **Enable auth middleware** — Uncomment auth protection in `middleware.ts`
2. **Rotate all exposed keys** — Every key in `.env.local` should be regenerated
3. **Add rate limiting** — Use `next-rate-limit` or Vercel's edge middleware
4. **Implement CSRF protection** — Add CSRF tokens to mutation endpoints
5. **Input sanitization** — Sanitize stored text against XSS

▫ Priority 2: Stability

6. **Fix corrupted filenames** — Clean up `route.tse.ts` artifacts
7. **Add error boundaries** — Wrap dynamic imports in React error boundaries
8. **Structured logging** — Replace `console.log/error` with Pino or Winston
9. **Health checks** — Add `/api/health` endpoint monitoring DB + AI connectivity

▫ Priority 3: Performance

10. **Parallelize clause analysis** — Process clauses concurrently

(Promise.allSettled)
11. **Cache structured_rules** — In-memory TTL cache (60min) for rule lookups
12. **Stream analysis progress** — Replace polling with Server-Sent Events
13. **Lazy load non-critical steps** — Skip graph/poison-pill for simple documents

🔵 Priority 4: Quality

14. **Add testing** — Unit tests for rule engine, integration tests for analysis pipeline
15. **E2E testing** — Playwright tests for upload → results flow
16. **Expand structured_rules** — More state-specific rules (currently focused on central laws)
17. **Model fine-tuning** — Fine-tune TF.js model on Indian contract corpus

⚪ Priority 5: Scale

18. **Queue system** — Move from inline analysis to Inngest/QStash queue
19. **Edge functions** — Move quick-scan to Vercel Edge for lower latency
20. **CDN for ML model** — Serve TF.js model from CDN for faster browser loading
21. **Database indexing** — Add composite indexes for common query patterns

---

> [!IMPORTANT]
> **ClauseWall is an extraordinarily ambitious and feature-rich project** — 765 source files, 30+ feature modules, 110+ API routes, and a sophisticated hybrid AI/database analysis engine. The architecture is sound and the feature set is genuinely innovative. The primary gap is security hardening and testing infrastructure needed for production deployment.